

Building this on Vespa — what transfers, and what to re-measure

This is a handover note for the team building library search on Vespa. It is not an argument about the engine: for a real service — hundreds of thousands of holdings, multiple branches, live circulation — Vespa is a reasonable choice and does several things this prototype had to hand-roll.

It is about everything the prototype **measured**, because that is the part a new engine does not give you and the part that is easy to lose in a rebuild. Some of it is counter-intuitive, and one finding runs directly against the thing Vespa makes easiest.

Every number here was produced by code in this repository and can be regenerated. Where a measurement is weak or contested, it says so.

1. What Vespa replaces, and what it does not decide

WHAT THE PROTOTYPE DOES	ON VESPA
BM25 in <code>src/bettersearch/keyword.py</code>	built in
Brute-force cosine over a numpy array	ANN with HNSW
Facet counting in <code>api/catalogue.py</code>	grouping
Fixed top-20 cut, then re-rank	phased ranking
<code>sentence-transformers</code> in the API process	an in-cluster embedder
A <code>pgvector</code> table as the production option	not needed

All of that is serving machinery, and Vespa does it better than we did. What Vespa does **not** decide — and what determined quality here — is the list below. Each has a measured answer, and each is a decision your schema and rank profiles will make implicitly whether or not anyone makes it deliberately:

- which encoder, and at what dimensionality
- what text actually gets embedded
- where the result list stops
- whether to fuse keyword and vector lanes, and how
- what to do about someone typing an exact title or author
- how you will know whether any of the above helped

2. The numbers, as an acceptance bar

41 queries over 4,000 real Open Library records, with 2,378 relevance judgements pooled from six retrieval lanes and judged blind. Each row names the encoder it used, because mixing them is how the gap gets misread:

ARM	NDCG@10
BM25 only	0.443
MiniLM 384d, semantic	0.579
RRF hybrid, BM25 + MiniLM	0.549 — below pure semantic
bge-base 768d, semantic	0.658 — what ships today
bge-base + LLM enrichment of thin records	0.743

A Vespa implementation over the same 4,000 records should reach **0.658** on semantic retrieval and **0.443** on BM25 alone. Below that, something in the port has gone wrong. Meaningfully above it, read §5 before celebrating.

With a reranking phase, the bar is 0.728, which is what the service now ships — a cross-encoder over the enriched text at depth 20, then exact-match promotion. That is the number to hold a Vespa global phase to; §5a has how it is built and the two ways we got it wrong first.

Both figures were re-measured while writing this note and reproduce exactly — 0.6582 and 0.4433, at 100% judged coverage — and again after the pool was widened to six lanes, unchanged. The bar is real, not remembered.

Why a bigger pool left them untouched is worth knowing before you read any nDCG@10 here: 28 of the 41 queries already have ten or more relevant records, so IDCG@10 is saturated and adding more cannot move the metric. **nDCG@10 is insensitive to pool growth on this eval; recall@10 is not**, since its denominator is every relevant record. That is why recall@10 is the headline wherever depth or coverage is in play, and it is why the enrichment and encoder figures above survived re-pooling while the reranking figures did not. The one arm of the table not re-measured after re-pooling is bge-base + enrichment (0.743) — its index was lost to a container restart and only the MiniLM enrichment arm was rebuilt, which did hold exactly.

The five deliberate known-item controls — exact titles and author names — score **1.000** today. They are a tripwire, not a target: if a change drops them, it has broken exact-name lookup, which is the one thing library users do most.

3. Four traps, each one hit or verified here

3.1 Hybrid is the easiest thing to switch on and the thing that lost

Reciprocal rank fusion of BM25 and vector scored **0.549 against pure semantic's 0.579** on this corpus. That result was initially explained away as an artefact of an eval set weighted towards keyword-hostile phrasing, with the prediction that real catalogue traffic — full of exact-name lookups — would be where fusion earned its place. It was re-tested on real records with five deliberate exact-name controls. It did not.

The mechanism is simple: blending two full rankings drags a good list down with a mostly-empty one, because BM25 returns nothing at all for a query that shares no term with any record, and a rank-based fusion still gives those empty positions weight.

This is **not** "do not use hybrid on Vespa". Vespa's phased ranking can express far better combinations than a flat RRF — a vector first phase with a BM25 signal among others in the second phase is a different

and more promising shape. It is: **do not ship a hybrid rank profile because the engine offers one.** Measure it against a vector-only profile on the same judgements first.

3.2 Pooling strategy will silently halve your quality

Vespa's Hugging Face embedder defaults to `pooling-strategy: mean`. The `bge` family is trained for **CLS** pooling. Both our server and our browser build use CLS.

Leave the default with a `bge` model and nothing errors. The service starts, the queries return records, the facets work, and retrieval is quietly worse.

This is worth dwelling on because the same *shape* of failure already cost this project a week: the browser build ran a different encoder from the server, and the only symptom was "the standalone gives different results". There was no error, no warning, and no test that could have caught it, because both sides were individually correct. Any mismatch between how documents and queries are encoded fails this way. Assume you will not notice it; add a check that would.

3.3 Circulation data must stay out of the embedded text

At real scale, availability, copies, due dates and branch change constantly. Bibliographic content does not.

Vespa supports partial updates to attribute fields that leave a tensor field on the same document untouched. Use that: **embed bibliographic content only**, and keep circulation state in attributes updated in place. If availability is part of the embedded text, every return and loan dirties a vector, and at a million holdings that is a re-embedding treadmill that will quietly become your largest compute cost.

The prototype arrives at the same place from a different direction: `content_hash` in `src/bettersearch/chunking.py` is taken over the embedded text only, so re-ingesting a corpus where only availability changed embeds nothing. That property is worth preserving explicitly in the schema rather than rediscovering.

3.5 Filter before the cut, never after

Vespa's filtered ANN does the right thing here, so this is a trap only if you reimplement the obvious order — which is what we did, and it was wrong.

Retrieving the top k by vector and *then* applying a facet does not search the filtered records. It keeps whichever of k already-chosen records happen to match. Measured over 41 queries $\times$ 10 facet values on 4,000 records:

	FILTER AFTER THE CUT	FILTER BEFORE IT
results returned	4.00	19.96
returned fewer than 3	31.2%	0.0%

Each of those facets covers 14–18% of the corpus. **Yours will be far narrower** — "available now, large print, at Ashcombe" is a fraction of a percent of a million holdings — and post-filtering would return an empty page nearly every time. The symptom is not bad ranking; it is a facet panel that appears broken.

Two consequences for the schema and the rank profile:

- Make the filter part of the query, not a step after it. In Vespa that is ordinary — attribute filters combine with `nearestNeighbor` and the engine handles the recall properly, including choosing between pre-

and post-filtering strategies. The thing to avoid is doing it in application code on a returned page.

- **Facet counts then have to come from somewhere wider than the page.** If they are counted over a filtered retrieval the sidebar collapses to the value already selected. Vespa grouping computes over the matched set, which is the right answer; just be deliberate about whether a count means "results you would get" or "matches in the corpus", because with a vector query those are very different numbers and only one of them is bounded.

3.4 Make the index carry its encoder's identity

Every index here stores the `model_id` it was built with and checks it on read *and* on write. Change the encoder and the index refuses to answer rather than scoring today's query against yesterday's vectors — which is the failure that returns confident, plausible, wrong results and raises nothing.

At a million records, re-embedding on a model change is a migration measured in hours, run alongside live traffic. The guard is what makes that survivable: whatever the Vespa equivalent is in your deployment — a field on the document, a rank profile that refuses an unexpected tensor dimension, a check in the feed pipeline — decide it before the first model change rather than during it.

4. What changes at real-library scale

The prototype is 4,000 records in one process. Things that did not matter there and will:

- **Cut depth becomes a phase boundary.** We cut the semantic list at a fixed top 20. That number was chosen by sweep — `scripts/tune_cutoff.py` — and fixed top-k beat every similarity threshold tried, because cosine has no absolute floor: on 4,000 records the threshold that looked right on 74 returned a median of 763 results. On Vespa this becomes `rerank-count` and the first phase/second phase split rather than a slice, but the finding stands: **do not cut a vector result list by score.**
- **Facets are counted before filtering.** Ours count over the whole ranked list so the sidebar shows what you *could* narrow to, not what survived the last click. Grouping makes this easy to get either way; it is a product decision worth making deliberately.
- **Multiple collections in one index.** We put 4,000 books and 114 events in one index and measured whether events were both findable and unobtrusive (`scripts/check_events.py`): 12/12 event-shaped queries reach an event, 0/5 known-item lookups show one. One index was enough. With more collection types — articles, archives, databases, e-resources — that balance needs re-checking, and rank fusion across collections is the lever if it tips.
- **Per-branch availability** is the obvious sharding and filtering question and the prototype says nothing useful about it. Ours is invented data.

5. How you will know you are better — and the trap in it

Read this before quoting any score, including ours.

The judgements were produced by pooling: the top 20 from each of six lanes — BM25, MiniLM over thin records, MiniLM over enriched records, bge-base, and the depth-200 `bge-reranker-base` lane plain and exact-match-promoted — unioned and judged blind. A record that no lane retrieved was never shown to a judge, and **an unjudged record counts as irrelevant.**

A Vespa implementation is a *seventh lane*. Every good record it finds that those six missed scores zero. **A better system can therefore score worse.**

Pool it before you quote it. `scripts/build_eval_set.py --lane-file label=rankings.json` takes `{query: [doc_id, ...]}` and judges only what no judge has seen — the last lane cost \$0.06 for 399 records. Carried grades are keyed on query *text*, not index, and the pool only ever grows, so this is safe to repeat.

This is not hypothetical and not small. It happened three times here:

- bge-base first measured *worse* than MiniLM, 0.578 against 0.588, and was nearly rejected. 100% of MiniLM's top 10 had been judged against **71%** of bge's. Re-pooling with bge as a fourth lane added 312 records no other lane had surfaced and moved it **+0.080** — the whole difference between "slightly worse" and "clearly better".
- A BM25 title-weighting change looked harmful for the same reason, and the sweep now reports coverage beside the score so nobody draws that conclusion again.
- Deep reranking measured at **-0.062** at rerank depth 200 on 59.5% coverage. The obvious fix — discard unjudged candidates, pin coverage at 100% — read **+0.087** and was believed. The judged truth is **+0.010**. Both estimators were wrong, in opposite directions, and the second was wrong by more. §5a has the detail; the lesson is that *correcting* for coverage is not the same as *having* the judgements.

And it is reproducible in two minutes, which is the clearest way to see it. Adding one more collection to the index — 114 community events alongside the 4,000 books — produces this:

INDEX	NDCG@10, SEMANTIC	JUDGED COVERAGE
4,000 books (the pooled corpus)	0.6582	100%
the same books + 114 events	0.6511	97.6%

The score fell by 0.007. But coverage fell too, because event records were never in the pool and every one that surfaces is scored as irrelevant by definition. Some of that 0.007 is the events genuinely displacing books, and some is an artefact of the judgements — **and this eval cannot tell you which**. On the keyword lane the same move takes coverage from 100% to 89.3%.

That is a 114-record change. A new engine over a real catalogue will move coverage far more than that.

So: `scripts/score_rankings.py` reports judged coverage next to every score.

```
python scripts/score_rankings.py vespa-semantic.json
python scripts/score_rankings.py ours-semantic.json vespa-semantic.json # compared
```

It takes the simplest thing any engine can emit — a JSON object mapping each query to its ranked `doc_ids` — and reports `nDCG@10`, `Recall@5`, `MRR@10`, the known-item controls, events intrusion, the weakest queries, and coverage. It uses the same metric functions as everything else in this repository, so a Vespa number and ours are the same arithmetic by construction rather than by agreement.

Reading the coverage line:

COVERAGE OF YOUR TOP 10	WHAT IT MEANS
≥ 90%	the comparison is fair; believe the score
75–90%	treat with caution; the score is probably an undercount
< 75%	not comparable — you are measuring the pool, not the engine

If coverage comes back low, the honest move is to re-pool and re-judge rather than accept the number. `scripts/build_eval_set.py` does this and takes a `--lane` per system, so Vespa can be added as a lane and the whole set re-judged for a few pounds of model time.

What it cannot tell you: anything about latency, cost, freshness, or behaviour at a scale 4,000 records cannot reach. It is a relevance bar and only that.

5a. Where the quality actually is — measured since this note was written

You asked whether Vespa could make meaning search better. It can, and the lever is the reranking phase. Here is the measurement, so the rank profile can be designed against numbers rather than hope.

The ceiling first. Reordering only the candidates we already retrieve, at depth 20 where judged coverage is 100%:

	NDCG@10
retrieval alone	0.6582
what we ship now — reranked at depth 20	0.7280
perfect reordering of the same 20 candidates	0.9124

The middle row is new since this note was first written. Reranking used to live only in a measurement script; it is now in `api/catalogue.py`, on by default, and the figure above is `run_search` scored against the judged queries rather than the script. So the acceptance bar in §2 has moved: a Vespa implementation should reach **0.658 on retrieval alone** and **0.728 with its global phase doing what ours does**.

The retrieval is fine. 161 judged-relevant records across 41 queries are retrieved and sitting at ranks 11–20. At depth 20 reranking never changes the candidate set, so §5's trap does not apply. **Deeper it does**, and §5's warning is not decoration — it cost us two wrong answers before we paid to re-pool. See *Depth* below.

What reached the ceiling, and what did not:

ARM	NDCG@10	CONTROLS	MS/QUERY
baseline	0.6582	1.000	0
BM25 as a weighted second phase	0.6782	1.000	0
cross-encoder over the records as they are	0.6573–0.6632	1.000	186–890
cross-encoder over enriched records	0.7149–0.7257	0.877–0.926	356–2,809
the same, plus exact-match promotion	0.7308	1.000	2,809
oracle	0.9124	1.000	—

Three things follow, and all three are rank-profile decisions:

1. **A cross-encoder is worth nothing on a bare catalogue record.** +0.005 and −0.001. The record is a 24-word metadata stub and a passage-reading model has nothing to read. The same model separates cleanly on a written sentence, so this is the data, not the model. **If you put a reranker in the global phase without giving it prose, you will pay the latency and get nothing.**
2. **Enrichment unlocks it.** At 95 words median the same reranker is worth +0.073. Reranking and enrichment are not alternatives; they are the same bet that these records are too thin, and they compound.
3. **It breaks exact-name lookup, and that must be caught.** Both enriched arms regressed the known-item controls — Sue Monk Kidd −0.369 — because a model reasoning over prose optimises aboutness at the cost of identity. A rule that lifts named records regardless of score restores them to 1.000 *and* raises the mean. Whatever you build, make the controls a gate in CI, not a thing somebody notices later.

The cost is the catch. `bge-reranker-base` is 2.8 s/query on CPU for 20 candidates. Vespa pays that on every search, so it needs ONNX, a GPU, or the MiniLM cross-encoder — which gets most of the gain for **13% of the time**: 96% of it on nDCG@10 (+0.070 against +0.073) and 89% on recall@10 (+0.058 against +0.066). If you ship one reranker, ship that one; it is the one we ship.

Two numbers, because they measure different things and the smaller one is the honest guide for a request path. The arms table above reports **356 ms** for MiniLM — that is `scripts/tune_reranking.py`'s per-arm cost. Measured warm through the serving path, the incremental cost of reranking a search is **104 ms median** on this container. Neither is Render's hardware and neither is yours; measure it on the node that will run it.

Two implementation details that cost us a cycle each, so you do not have to. Reranking must run **before** exact-match promotion — reversed, it pushes the named records straight back down and the controls land at 0.877. And a cross-encoder's output must not reach the response as a relevance score: it is an uncalibrated logit, not a cosine, and anything downstream reading that field will be quietly wrong. We keep the retrieval score and label the card instead.

Depth: set rerank-count to 20. This is the headline correction to an earlier draft of this note, which told you quality does not fall off with depth. It does, monotonically:

RERANK DEPTH	20	50	100	200
recall@10 vs baseline	+0.066	+0.047	+0.035	+0.010
judged coverage	100%	92.7%	98.8%	100%
ms/query (CPU)	2,809	5,548	10,093	19,937

Seven times the latency for a seventh of the gain. Rerank the twenty you display.

How that draft got it wrong is the most useful thing in this section, because it is a trap any team measuring a Vespa rank profile will meet. The first sweep, pooled from four lanes, read `+0.079 / +0.029 / -0.007 / -0.062` with coverage falling to 59.5% — §5's bias, clearly. So it was re-run with unjudged candidates *discarded*, pinning coverage at 100%, which read `+0.079 / +0.072 / +0.091 / +0.087` and was written up here as the true shape.

That correction was the bigger error. Discarding unjudged candidates does not remove the bias, it inverts it: what gets discarded is exactly the junk the reranker promoted and nobody graded, so the arm is scored only where it was already known to be doing well. It overstated the gain roughly ninefold at depth 200 — `+0.087` against a judged `+0.010`.

Neither estimator is safe. Only judgements are. We pooled the depth-200 reranked lane and graded the 399 records it surfaced that no judge had seen, for \$0.06; 81 were relevant; coverage went to 100% and the number stopped moving. `data/rankings/` holds that lane so you can reproduce the pool, and `scripts/bound_repool_effect.py` bounds the gap before you spend — read its ordering, not its margins, which were 3× out.

Still unexplained: the best arm captures 29% of the available headroom. The remaining 0.18 is real, measured, and nobody here knows what reaches it. That is the most interesting open question in this handover, and the first one worth spending a week on.

Reproduce with `python scripts/tune_reranking.py`.

6. Things worth not rebuilding

Two small pieces earned their place for reasons that are not about ranking, and are cheap to carry over:

- **Exact-match promotion.** A model-free pass over title and author that catches what neither lane can see — word order. It is worth almost nothing as ranking (nDCG 0.658 either way) and ships anyway, because it makes a *guarantee*: a record whose exact title someone typed is on page one because a rule says so, not because the encoder agreed. It also gives the card something true to say — "by this author" is legible in a way a cosine score is not.
- **Saying why a record is there.** Keyword results name the query words the record does *not* contain; meaning-based results name the record's own subject headings closest to the query. Both are cheap, and they are most of what makes a meaning-based result list feel trustworthy rather than arbitrary.

7. Open questions for your team

1. What is the actual corpus — holdings only, or articles, archives and e-resources too? The one-index finding was measured on two collections, not five.
2. Is enrichment of thin records on the table? It is worth +0.085 nDCG on top of the best encoder, and it is the second thing to try, not the first — a bigger encoder buys most of what it buys, for free.
3. What is the re-embedding plan when the encoder changes, and who notices if documents and queries fall out of step?
4. Will you re-pool the judgements with Vespa as a lane before comparing? If not, §5 says what the comparison is worth.
5. Multilingual? Every measurement here is English-only and the encoder choice would change.